

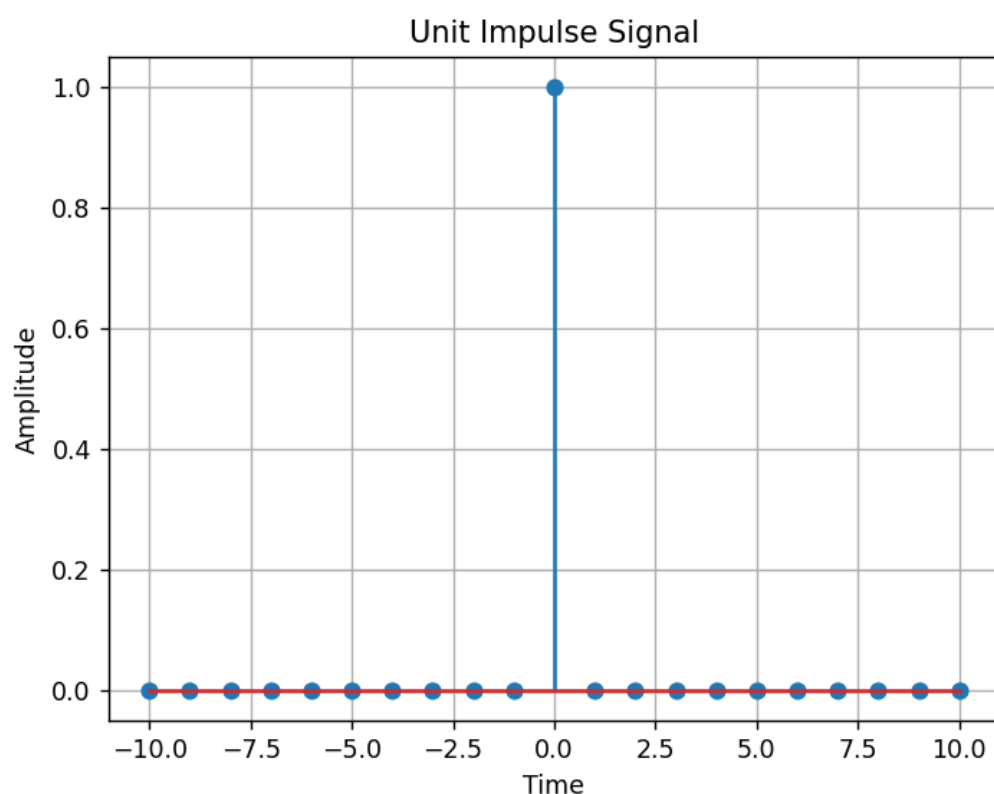
 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital signal and Image processing(01CT1513)	Aim : simulate discrete time sequence	
Experiment no :-1	Date: 15-07-2026	Enrolment No: 92400133131

```

Code 1 :- import numpy as np
import matplotlib.pyplot as plt
def unit_impulse(length, position):
    signal = np.zeros(length);
    signal[position] = 1
    return signal
# Parameters
start = -10 # Start value of the x-axis range
stop = 10 # Stop value of the x-axis range
step = 1 # Step size
# Generate x-axis values
x = np.arange(start, stop+step, step)
# Generate unit impulse signal
impulse_signal = unit_impulse(len(x), abs(start)//step)
# Plot the signal
plt.stem(x, impulse_signal)
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.title('Unit Impulse Signal')
plt.grid(True)
plt.show()

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital signal and Image processing(01CT1513)	Aim : simulate discrete time sequence	
Experiment no :-1	Date: 15-07-2026	Enrolment No: 92400133131



Code 2

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_impulse_train(signal_length, period):
    impulse_train = np.zeros(signal_length)
    for n in range(signal_length):
        if n % period == 0:
            impulse_train[n] = 1
    return impulse_train

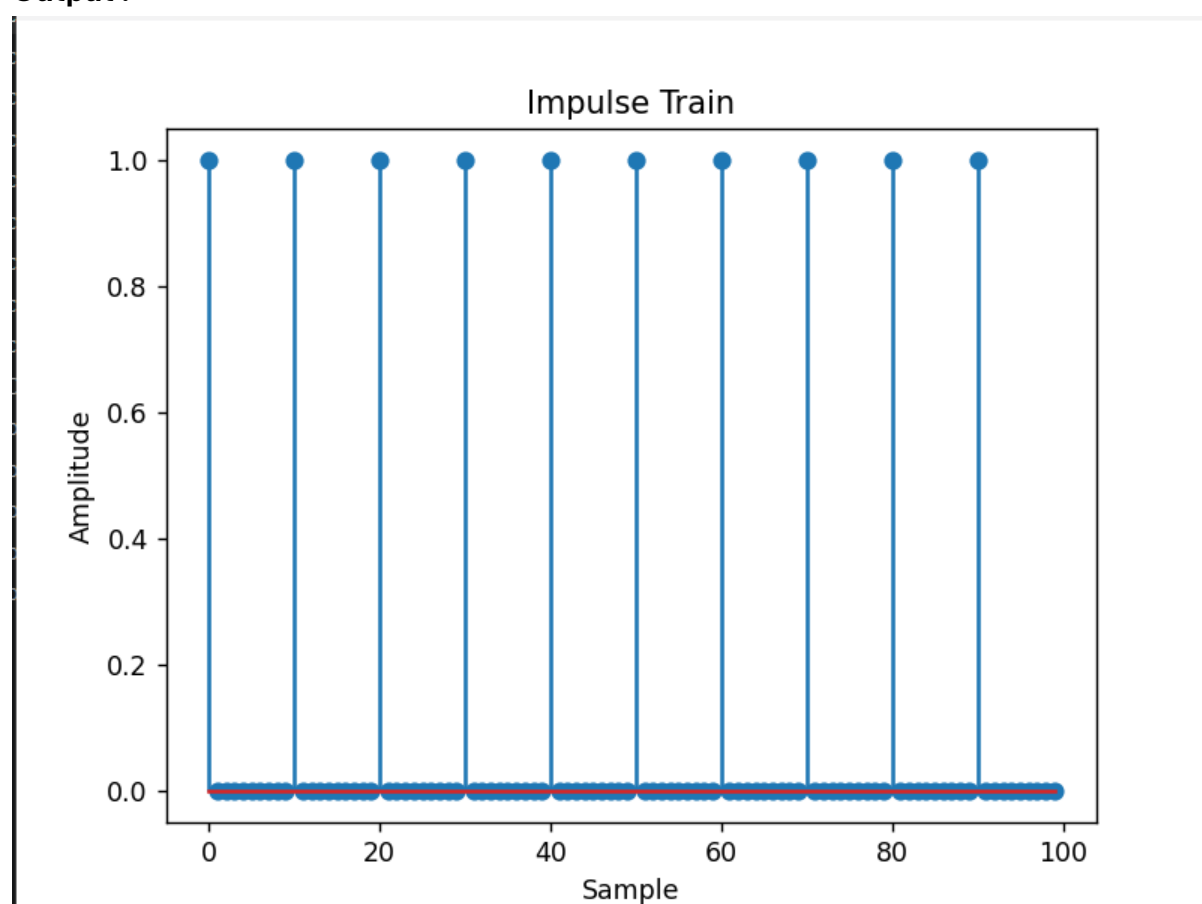
signal_length = 100
period = 10
impulse_train = simulate_impulse_train(signal_length, period)

plt.stem(impulse_train)
plt.title('Impulse Train')
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital signal and Image processing(01CT1513)	Aim : simulate discrete time sequence	
Experiment no :-1	Date: 15-07-2026	Enrolment No: 92400133131

```
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.show()
```

Output :-



```
Code 3:- import numpy as np
import matplotlib.pyplot as plt

def simulate_continuous_unit_step(time):
    unit_step = np.zeros_like(time)
    unit_step[time >= 0] = 1
    return unit_step

def simulate_discrete_unit_step(num_samples):
    unit_step = np.zeros(num_samples)
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital signal and Image processing(01CT1513)	Aim : simulate discrete time sequence	
Experiment no :-1	Date: 15-07-2026	Enrolment No: 92400133131

```

unit_step[num_samples // 21:] = 1
return unit_step

time = np.linspace(-5, 5, 1000)

continuous_unit_step = simulate_continuous_unit_step(time)

num_samples = 20

discrete_unit_step = simulate_discrete_unit_step(num_samples)

plt.figure(figsize=(10, 6))

plt.subplot(2, 1, 1)
plt.plot(time, continuous_unit_step, linewidth=2)
plt.title("Continuous Unit Step Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)

plt.subplot(2, 1, 2)
plt.stem(range(num_samples), discrete_unit_step)
plt.title("Discrete Unit Step Signal")
plt.xlabel("Sample")
plt.ylabel("Amplitude")
plt.grid(True)

plt.tight_layout()
plt.show()

```



Marwadi
University

Marwadi University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Digital signal and
Image
processing(01CT1513)**

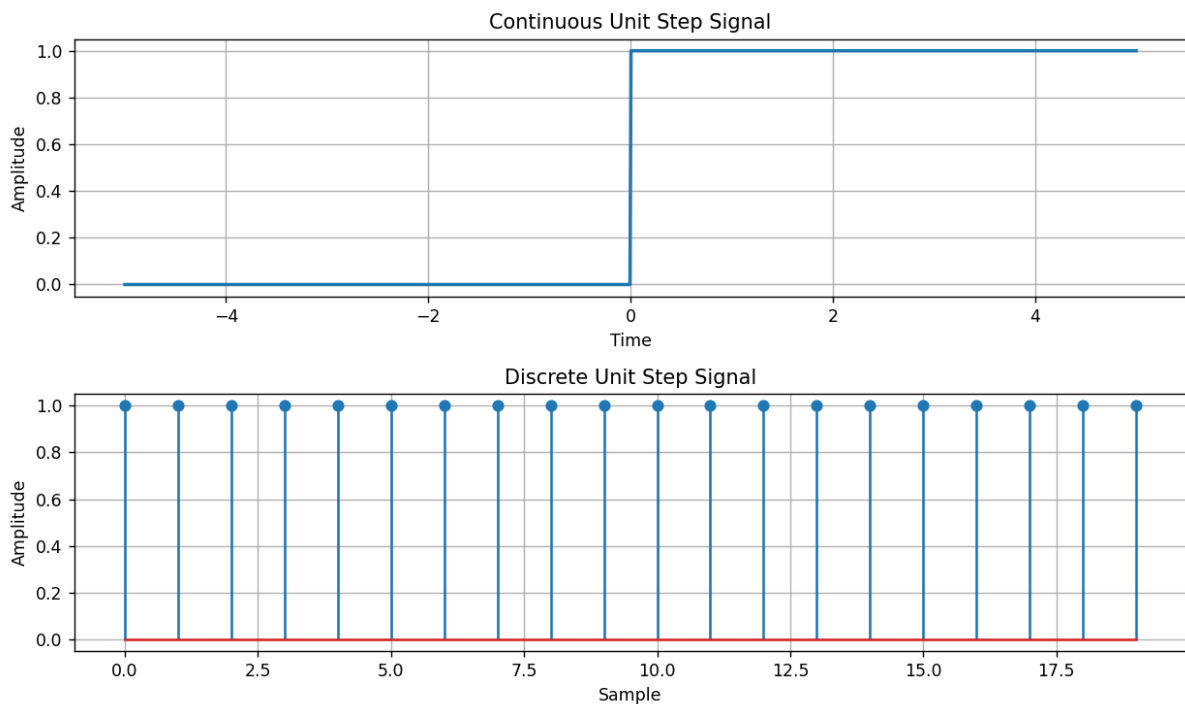
Aim : simulate discrete time sequence

Experiment no :-1

Date: 15-07-2026

Enrolment No: 92400133131

Output :-



```
code 4 :- import numpy as np
import matplotlib.pyplot as plt
import time

def simulate_continuous_ramp(time, slope):
    ramp = np.zeros_like(time)
    ramp[time >= 0] = slope * time[time >= 0]
    return ramp

def simulate_discrete_ramp(num_samples, slope):
    ramp = np.zeros(num_samples)
    ramp[num_samples // 2:] = slope * np.arange(num_samples // 2,
num_samples)
    return ramp

# Define the time range for the continuous ramp signal
time = np.linspace(-5, 5, 1000) # Time range from -5 to 5
# Define the number of samples and slope for the discrete ramp signal
num_samples = 20 # Number of samples
slope = 2 # Slope of the ramp
```

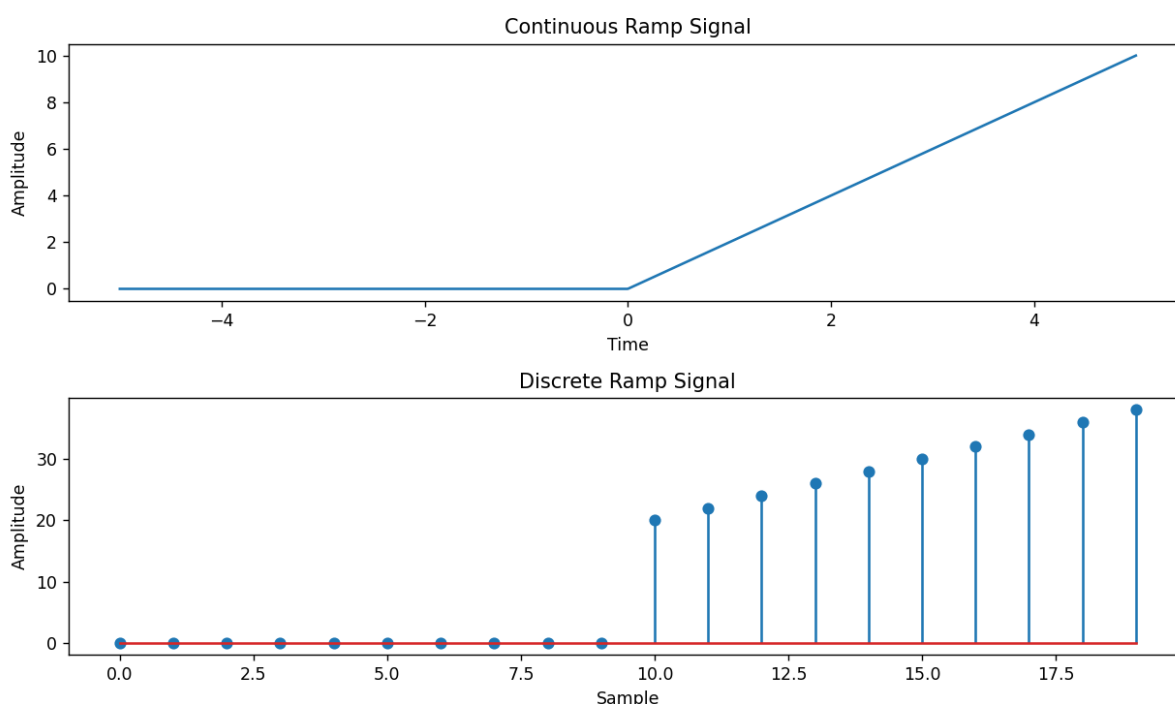
 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital signal and Image processing(01CT1513)	Aim : simulate discrete time sequence	
Experiment no :-1	Date: 15-07-2026	Enrolment No: 92400133131

```

# Simulate the continuous ramp signal
continuous_ramp = simulate_continuous_ramp(time, slope)
# Simulate the discrete ramp signal
discrete_ramp = simulate_discrete_ramp(num_samples, slope)
# Plot and display the continuous and discrete ramp signals
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_ramp)
plt.title('Continuous Ramp Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_ramp)
plt.title('Discrete Ramp Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()

```

Output :-



 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital signal and Image processing(01CT1513)	Aim : simulate discrete time sequence	
Experiment no :-1	Date: 15-07-2026	Enrolment No: 92400133131

```

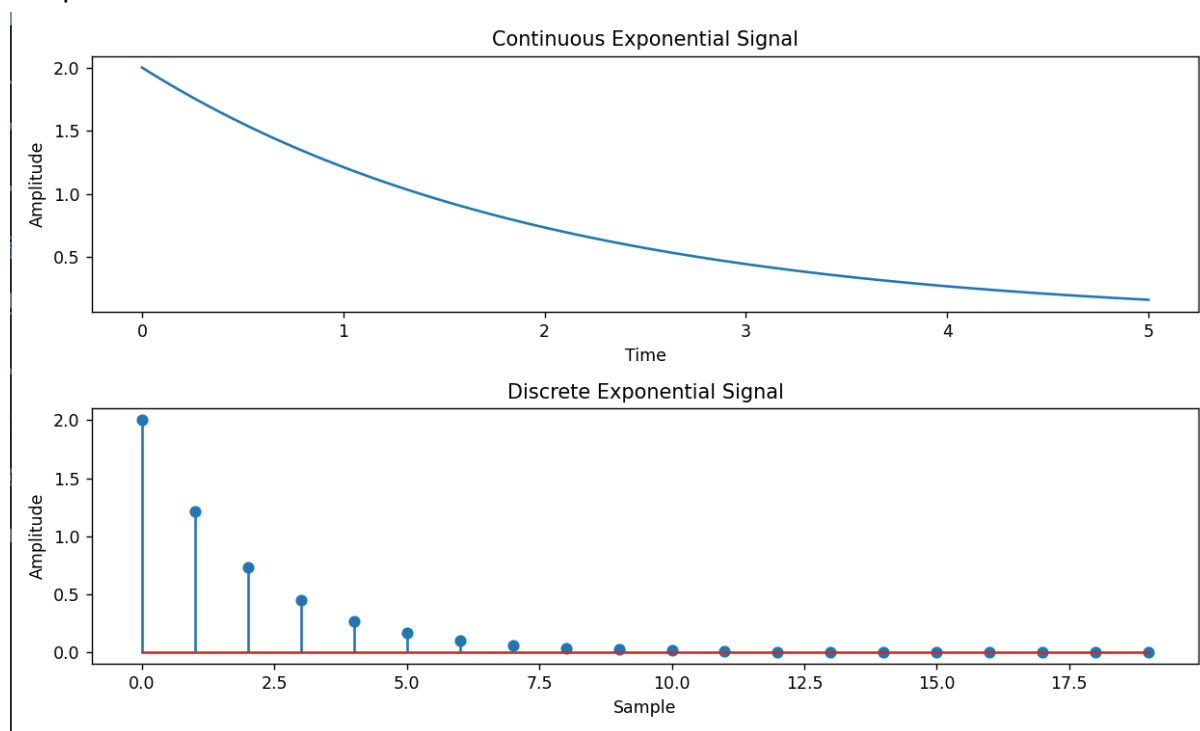
Code 5:-import numpy as np
import matplotlib.pyplot as plt
import time
def simulate_continuous_exponential(time, amplitude, coefficient):
    exponential_signal = amplitude * np.exp(coefficient * time)
    return exponential_signal
def simulate_discrete_exponential(num_samples, amplitude, coefficient):
    exponential_signal = amplitude * np.exp(coefficient *
np.arange(num_samples))
    return exponential_signal
# Define the time range for the continuous exponential signal
time = np.linspace(0, 5, 1000) # Time range from 0 to 5
# Define the number of samples, initial amplitude, and coefficient for the
discrete exponential

num_samples = 20 # Number of samples
amplitude = 2 # Initial amplitude
coefficient = -0.5 # Exponential coefficient
# Simulate the continuous exponential signal
continuous_exponential = simulate_continuous_exponential(time, amplitude,
coefficient)
# Simulate the discrete exponential signal
discrete_exponential = simulate_discrete_exponential(num_samples, amplitude,
coefficient)
# Plot and display the continuous and discrete exponential signals
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_exponential)
plt.title('Continuous Exponential Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_exponential)
plt.title('Discrete Exponential Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital signal and Image processing(01CT1513)	Aim : simulate discrete time sequence	
Experiment no :-1	Date: 15-07-2026	Enrolment No: 92400133131

Output :-

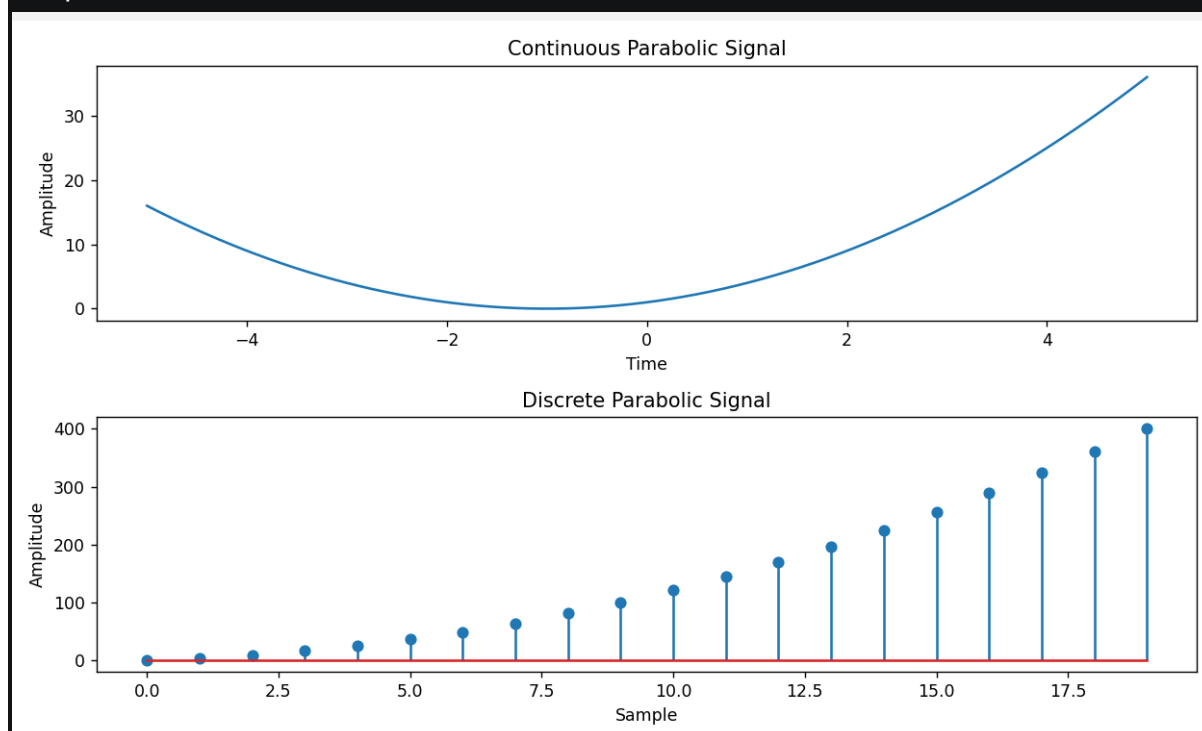


```
Code 6 :-import numpy as np
import matplotlib.pyplot as plt
def simulate_continuous_parabolic(time, coefficients):
    parabolic_signal = np.polyval(coefficients, time)
    return parabolic_signal
def simulate_discrete_parabolic(num_samples, coefficients):
    parabolic_signal = np.polyval(coefficients, np.arange(num_samples))
    return parabolic_signal
# Define the time range for the continuous parabolic signal
time = np.linspace(-5, 5, 1000) # Time range from -5 to 5
# Define the number of samples and coefficients for the discrete parabolic signal
num_samples = 20 # Number of samples
coefficients = [1, 2, 1] # Coefficients of the parabolic signal
# Simulate the continuous parabolic signal
continuous_parabolic = simulate_continuous_parabolic(time, coefficients)
# Simulate the discrete parabolic signal
discrete_parabolic = simulate_discrete_parabolic(num_samples, coefficients)
# Plot and display the continuous and discrete parabolic signals
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital signal and Image processing(01CT1513)	Aim : simulate discrete time sequence	
Experiment no :-1	Date: 15-07-2026	Enrolment No: 92400133131

```
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_parabolic)
plt.title('Continuous Parabolic Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_parabolic)
plt.title('Discrete Parabolic Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
```

Output :-



code 7 :-

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_continuous_sine_wave(time, amplitude, frequency, phase):
    sine_wave = amplitude * np.sin(2 * np.pi * frequency * time + phase)
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital signal and Image processing(01CT1513)	Aim : simulate discrete time sequence	
Experiment no :-1	Date: 15-07-2026	Enrolment No: 92400133131

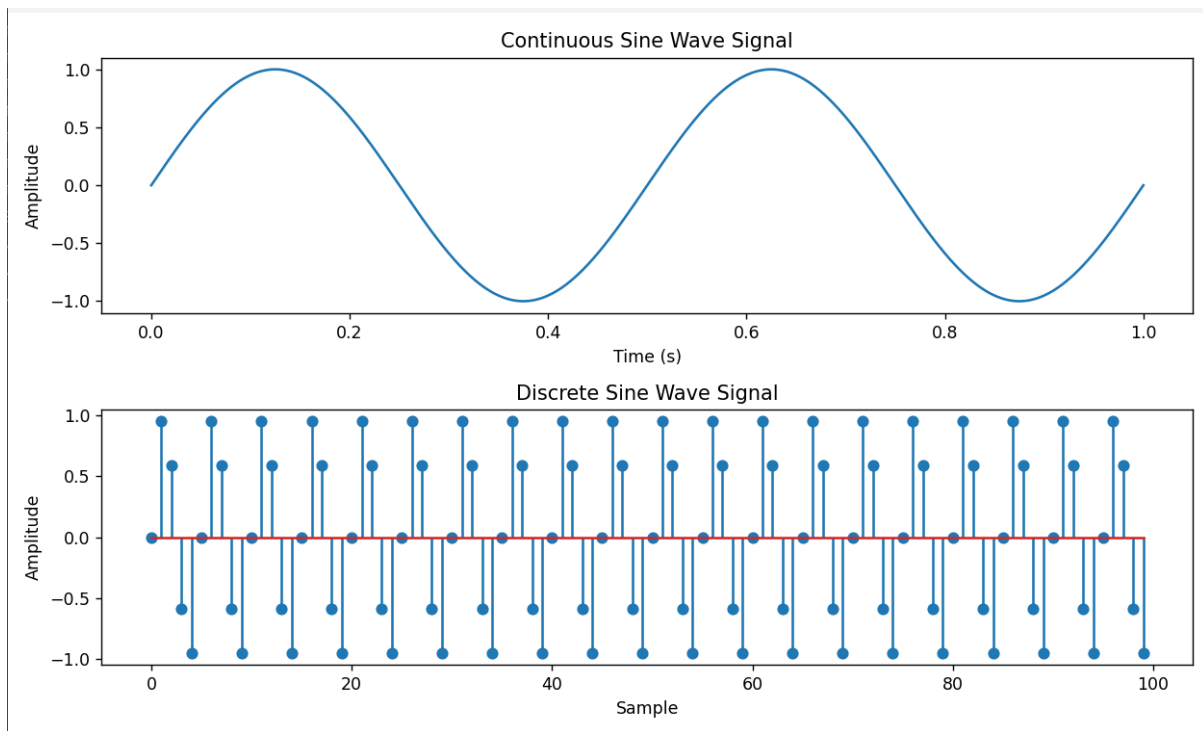
```

    return sine_wave
def simulate_discrete_sine_wave(num_samples, sampling_frequency, amplitude,
frequency,
phase):
    time = np.arange(num_samples) / sampling_frequency
    sine_wave = amplitude * np.sin(2 * np.pi * frequency * time + phase)
    return sine_wave
# Define the time range for the continuous sine wave signal
time = np.linspace(0, 1, 1000) # Time range from 0 to 1 second
# Define the
num_samples = 100 # Number of samples
sampling_frequency = 10 # Sampling frequency in Hz
amplitude = 1 # Amplitude of the sine wave
frequency = 2 # Frequency of the sine wave in Hz
phase = 0 # Phase angle of the sine wave in radians
# Simulate the continuous sine wave signal
continuous_sine_wave = simulate_continuous_sine_wave(time, amplitude,
frequency, phase)
# Simulate the discrete sine wave signal
discrete_sine_wave = simulate_discrete_sine_wave(num_samples,
sampling_frequency,
amplitude, frequency, phase)
# Plot and display the continuous and discrete sine wave signals
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_sine_wave)
plt.title('Continuous Sine Wave Signal')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_sine_wave)
plt.title('Discrete Sine Wave Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital signal and Image processing(01CT1513)	Aim : simulate discrete time sequence	
Experiment no :-1	Date: 15-07-2026	Enrolment No: 92400133131

Output :-



```
Code 8:-import numpy as np
import matplotlib.pyplot as plt

def simulate_function(time):
    y = np.zeros_like(time)
    y[time >= 0] = 1
    y[time >= 1] += 1
    y[time >= -5] += 3
    return y

time = np.linspace(-10, 10, 1000)

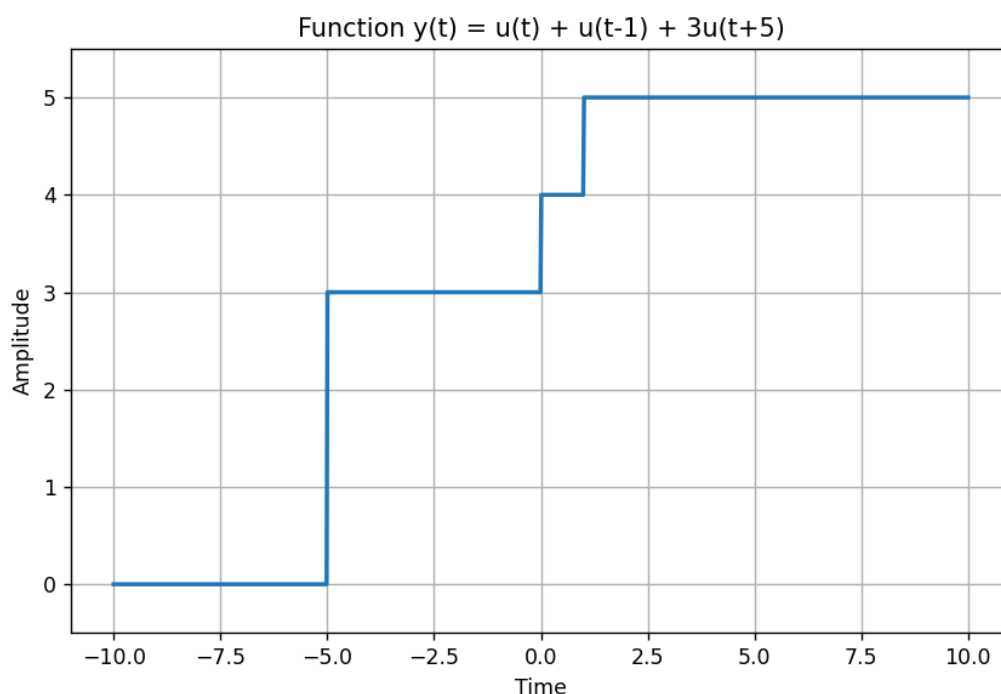
function_values = simulate_function(time)

plt.figure(figsize=(8, 5))
plt.plot(time, function_values, linewidth=2)
plt.title("Function y(t) = u(t) + u(t-1) + 3u(t+5)")
plt.xlabel("Time")
plt.ylabel("Amplitude")
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital signal and Image processing(01CT1513)	Aim : simulate discrete time sequence	
Experiment no :-1	Date: 15-07-2026	Enrolment No: 92400133131

```
plt.ylim([-0.5, 5.5])
plt.grid(True)
plt.show()
```

Output :-

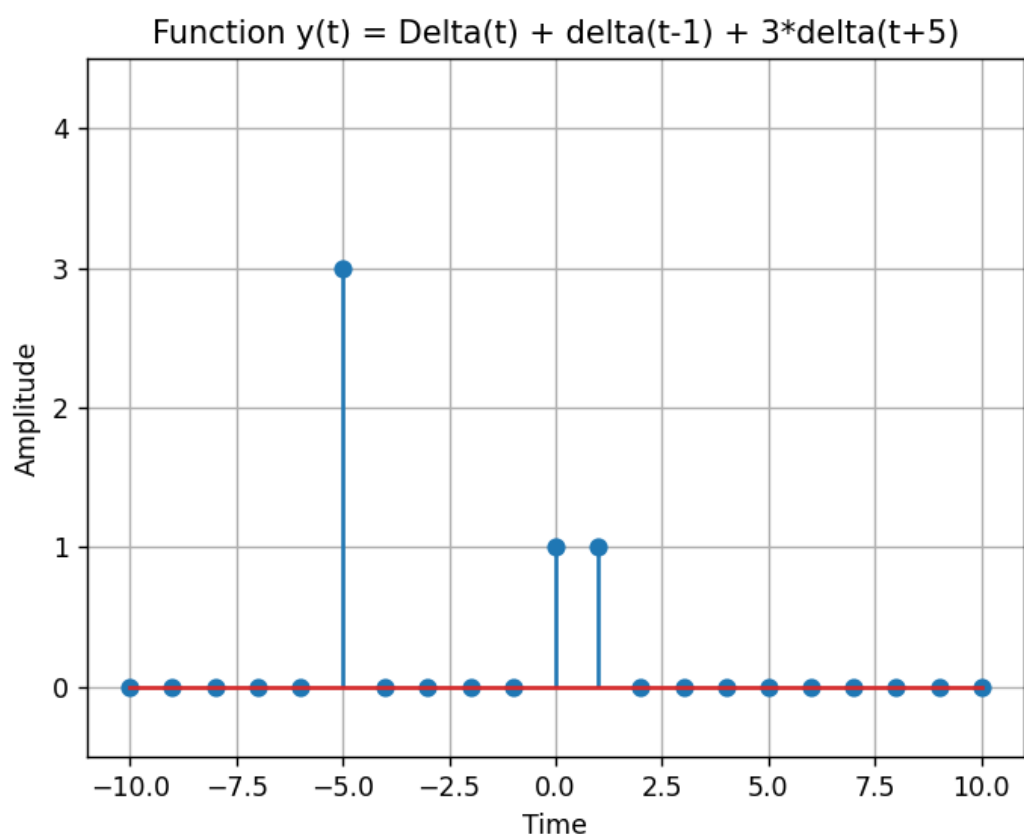


```
Code 9:- import numpy as np
import matplotlib.pyplot as plt
def simulate_function(time):
    y = np.zeros_like(time)
    y[time == 0] = 1
    y[time == 1] += 1
    y[time == -5] += 3
    return y
# Define the time range
time = np.arange(-10, 11)
# Simulate the function
function_values = simulate_function(time)
# Plot and display the function
plt.stem(time, function_values)
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital signal and Image processing(01CT1513)	Aim : simulate discrete time sequence	
Experiment no :-1	Date: 15-07-2026	Enrolment No: 92400133131

```
plt.title('Function y(t) = Delta(t) + delta(t-1) + 3*delta(t+5)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.ylim([-0.5, 4.5])
plt.grid(True)
plt.show()
```

Output :-



Conclusion :- The simulation of discrete-time sequences demonstrated how signals are represented and analyzed at distinct time intervals. Different types of sequences, such as unit step, impulse, ramp, and exponential sequences, were successfully generated and observed. The experiment helped in understanding the behavior and

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital signal and Image processing(01CT1513)	Aim : simulate discrete time sequence	
Experiment no :-1	Date: 15-07-2026	Enrolment No: 92400133131

characteristics of discrete-time signals in signal processing. Overall, the simulation provided practical knowledge of sequence generation and reinforced the theoretical concepts of discrete-time systems.